

RoboPet — Phase 1.5

Visual Polish & Flame Integration (Cyberpunk HUD)

Goal: fill the empty center with a live Flame canvas (an animated, breathing robot placeholder) and re-skin the PetScreen into a futuristic cyberpunk HUD -- glassmorphism stat panels, neon glowing borders, and tech fonts. All Riverpod logic, the stream, and the Feed / Oil / Play actions stay exactly the same -- this is purely presentation.

1. Dependencies

Add Flame (the 2D engine) and Google Fonts (tech typography) to pubspec.yaml:

```
dependencies:
  flame: ^1.18.0
  google_fonts: ^6.2.1
```

New / changed files:

```
lib/
  game/
    robot_game.dart      # NEW -- Flame canvas + animated robot + RobotStage
  ui/
    theme.dart           # NEW -- cyberpunk palette, fonts, GlassPanel
    pet_screen.dart      # REWRITTEN -- HUD layout (same providers/actions)
    main.dart            # CHANGED -- use buildCyberpunkTheme()
```

2. The Flame canvas

lib/game/robot_game.dart

A transparent FlameGame (so the HUD gradient shows through) hosting a glowing platform + a robot that breathes (scale sine) and bobs (vertical sine). RobotStage creates the game ONCE so the frequent Riverpod rebuilds never recreate it.

```
import 'dart:math' as math;

import 'package:flame/components.dart';
import 'package:flame/game.dart';
import 'package:flutter/material.dart';

/// A lightweight Flame canvas hosting the robot avatar. For now it renders an
/// animated (breathing + bobbing) placeholder; swap in real sprites later.
class RobotGame extends FlameGame {
  RobotGame({this.accent = const Color(0xFF22E1FF)});

  final Color accent;
```

```

@Override
Color backgroundColor() => const Color(0x00000000); // transparent

@Override
Future<void> onLoad() async {
    await super.onLoad();
    add(GlowPlatform());
    add(RobotAvatar());
}
}

/// Soft glowing pad the robot stands on.
class GlowPlatform extends PositionComponent with HasGameReference<RobotGame> {
    @Override
    void onLoad() {
        anchor = Anchor.center;
        size = Vector2(200, 46);
        position = Vector2(game.size.x / 2, game.size.y / 2 + 96);
    }

    @Override
    void onGameResize(Vector2 s) {
        super.onGameResize(s);
        position = Vector2(s.x / 2, s.y / 2 + 96);
    }

    @Override
    void render(Canvas canvas) {
        final rect = Rect.fromLTWH(0, 0, size.x, size.y);
        canvas.drawOval(
            rect,
            Paint()
                ..shader = RadialGradient(
                    colors: [game.accent.withOpacity(0.5), Colors.transparent],
                ).createShader(rect),
        );
    }
}

/// The breathing / bobbing robot placeholder.
class RobotAvatar extends PositionComponent with HasGameReference<RobotGame> {
    double _t = 0;

    @Override
    void onLoad() {
        anchor = Anchor.center;
        size = Vector2(150, 150);
        position = game.size / 2;
    }

    @Override
    void onGameResize(Vector2 s) {
        super.onGameResize(s);
        position = s / 2;
    }

    @Override
    void update(double dt) {
        super.update(dt);
        _t += dt;
        scale = Vector2.all(1 + 0.045 * math.sin(_t * 2.2)); // breathe
        position = game.size / 2 + Vector2(0, 7 * math.sin(_t * 1.6)); // bob
    }

    @Override
    void render(Canvas canvas) {
        final w = size.x;
        final h = size.y;
        final accent = game.accent;

```

```

final body = RRect.fromRectAndRadius(
  Rect.fromLTWH(w * 0.16, h * 0.22, w * 0.68, h * 0.6),
  const Radius.circular(30),
);

// Outer glow.
canvas.drawRRect(
  body,
  Paint()
    ..color = accent.withOpacity(0.45)
    ..maskFilter = const MaskFilter.blur(BlurStyle.normal, 20),
);
// Body fill.
canvas.drawRRect(
  body,
  Paint()
    ..shader = const LinearGradient(
      begin: Alignment.topCenter,
      end: Alignment.bottomCenter,
      colors: [Color(0xFF1B2740), Color(0xFF0C1322)],
    ).createShader(body.outerRect),
);
// Neon border.
canvas.drawRRect(
  body,
  Paint()
    ..style = PaintingStyle.stroke
    ..strokeWidth = 2.2
    ..color = accent,
);

// Antenna.
final pen = Paint()
  ..color = accent
  ..strokeWidth = 3
  ..strokeCap = StrokeCap.round;
canvas.drawLine(Offset(w * 0.5, h * 0.22), Offset(w * 0.5, h * 0.08), pen);
canvas.drawCircle(Offset(w * 0.5, h * 0.06), 5, Paint()..color = accent);

// Eyes (glow + white core).
final eyeY = h * 0.46;
final glow = Paint()
  ..color = accent
  ..maskFilter = const MaskFilter.blur(BlurStyle.normal, 4);
for (final dx in [0.40, 0.60]) {
  canvas.drawCircle(Offset(w * dx, eyeY), 9, glow);
  canvas.drawCircle(Offset(w * dx, eyeY), 5, Paint()..color = Colors.white);
}

// Mouth.
canvas.drawRRect(
  RRect.fromRectAndRadius(
    Rect.fromLTWH(w * 0.40, h * 0.62, w * 0.20, h * 0.04),
    const Radius.circular(4),
  ),
  Paint()..color = accent.withOpacity(0.8),
);
}
}

/// Stable host widget: builds the game ONCE so UI rebuilds don't recreate it.
class RobotStage extends StatefulWidget {
  const RobotStage({super.key, this.accent = const Color(0xFF22E1FF)});
  final Color accent;

  @override
  State<RobotStage> createState() => _RobotStageState();
}

```

```

class _RobotStageState extends State<RobotStage> {
  late final RobotGame _game = RobotGame(accent: widget.accent);

  @override
  Widget build(BuildContext context) => GameWidget(game: _game);
}

```

When you have sprites, replace `RobotAvatar.render` with a `SpriteAnimationComponent` (e.g. `await game.loadSprite(robot.png)`) -- the rest of the stage and HUD stay identical.

3. Theme, palette & the glass panel

lib/ui/theme.dart

```

import 'dart:ui';

import 'package:flutter/material.dart';
import 'package:google_fonts/google_fonts.dart';

class AppColors {
  static const bg = Color(0xFF05060A);
  static const surface = Color(0xFF0E1320);
  static const cyan = Color(0xFF22E1FF);
  static const magenta = Color(0xFFFFF3CAC);
  static const green = Color(0xFF36F1A6);
  static const amber = Color(0xFFFFFC24B);
  static const pink = Color(0xFFFFF5C8A);
}

ThemeData buildCyberpunkTheme() {
  final base = ThemeData(brightness: Brightness.dark, useMaterial3: true);
  return base.copyWith(
    scaffoldBackgroundColor: AppColors.bg,
    colorScheme: base.colorScheme.copyWith(
      primary: AppColors.cyan,
      secondary: AppColors.magenta,
      surface: AppColors.surface,
    ),
    textTheme: GoogleFonts.rajdhaniTextTheme(base.textTheme).copyWith(
      headlineSmall: GoogleFonts.orbitron(
        fontSize: 22,
        fontWeight: FontWeight.w700,
        color: Colors.white,
        letterSpacing: 1.5,
      ),
      titleSmall: GoogleFonts.orbitron(
        fontSize: 12,
        fontWeight: FontWeight.w600,
        letterSpacing: 1.2,
        color: Colors.white70,
      ),
    ),
  );
}

/// Reusable glassmorphism container: blurred translucent fill + glowing border.
class GlassPanel extends StatelessWidget {
  const GlassPanel({
    super.key,
    required this.child,
  }) : super();
}

```

```

    this.padding = const EdgeInsets.all(16),
    this.radius = 18,
    this.blur = 14,
    this.borderColor = Colors.white24,
    this.glow = false,
  });

  final Widget child;
  final EdgeInsets padding;
  final double radius;
  final double blur;
  final Color borderColor;
  final bool glow;

  @override
  Widget build(BuildContext context) {
    return ClipRRect(
      borderRadius: BorderRadius.circular(radius),
      child: BackdropFilter(
        filter: ImageFilter.blur(sigmaX: blur, sigmaY: blur),
        child: Container(
          padding: padding,
          decoration: BoxDecoration(
            borderRadius: BorderRadius.circular(radius),
            gradient: LinearGradient(
              begin: Alignment.topLeft,
              end: Alignment.bottomRight,
              colors: [
                Colors.white.withOpacity(0.08),
                Colors.white.withOpacity(0.02),
              ],
            ),
          ),
          border: Border.all(color: borderColor, width: 1.2),
          boxShadow: glow
            ? [
                BoxShadow(
                  color: borderColor.withOpacity(0.4),
                  blurRadius: 18,
                  spreadRadius: -2,
                ),
              ],
            : null,
        ),
      ),
    );
  }
}

```

4. The redesigned Pet Screen

lib/ui/pet_screen.dart

Same activeRobotStreamProvider consumption and same repo.feed/clean/play calls -- only the visuals changed. Layout is a Stack: animated grid background -> Flame stage in the center -> glass HUD panels on top.

```

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../app/providers.dart';

```

```

import '../data/models/robot_instance.dart';
import '../game/robot_game.dart';
import 'theme.dart';

class PetScreen extends ConsumerWidget {
  const PetScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final asyncRobot = ref.watch(activeRobotStreamProvider);

    return Scaffold(
      body: asyncRobot.when(
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, _) => Center(child: Text('Error: $e')),
        data: (robot) => robot == null
          ? const Center(child: Text('No active robot'))
          : _PetView(robot: robot),
      ),
    );
  }
}

class _PetView extends ConsumerWidget {
  const _PetView({required this.robot});
  final RobotInstance robot;

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final repo = ref.read(robotRepositoryProvider);

    return Stack(
      children: [
        const Positioned.fill(child: _HudBackground()),
        SafeArea(
          child: Padding(
            padding: const EdgeInsets.fromLTRB(18, 10, 18, 18),
            child: Column(
              children: [
                _Header(robot: robot),
                Expanded(child: RobotStage(accent: AppColors.cyan)),
                _StatCluster(robot: robot),
                const SizedBox(height: 16),
                Row(
                  children: [
                    Expanded(
                      child: _NeonButton(
                        label: 'Feed',
                        icon: Icons.bolt,
                        color: AppColors.green,
                        onTap: () => repo.feed(robot.instanceId),
                      ),
                    ),
                    const SizedBox(width: 12),
                    Expanded(
                      child: _NeonButton(
                        label: 'Oil',
                        icon: Icons.water_drop,
                        color: AppColors.amber,
                        onTap: () => repo.clean(robot.instanceId),
                      ),
                    ),
                    const SizedBox(width: 12),
                    Expanded(
                      child: _NeonButton(
                        label: 'Play',
                        icon: Icons.sports_esports,
                        color: AppColors.pink,
                        onTap: () => repo.play(robot.instanceId),
                      ),
                    ),
                  ],
                ),
              ],
            ),
          ),
        ),
      ],
    );
  }
}

```

```

    ),
  ],
),
],
),
);
}
}

class _Header extends StatelessWidget {
  const _Header({required this.robot});
  final RobotInstance robot;

  @override
  Widget build(BuildContext context) {
    final text = Theme.of(context).textTheme;
    return Row(
      children: [
        Expanded(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(robot.nickname.toUpperCase(), style: text.headlineSmall),
              const SizedBox(height: 2),
              Text(
                '${robot.rarity.name.toUpperCase()} • ',
                '${robot.robotClass.name.toUpperCase()}',
                style: text.bodySmall?.copyWith(
                  color: AppColors.cyan,
                  letterSpacing: 2,
                ),
              ),
            ],
          ),
        ),
      ],
    ),
    GlassPanel(
      padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 8),
      borderColor: AppColors.cyan.withOpacity(0.6),
      glow: true,
      child: Text('LV ${robot.level}',
        style: text.titleSmall?.copyWith(color: Colors.white)),
    ),
    if (robot.neglectStrikes > 0) ...[
      const SizedBox(width: 10),
      GlassPanel(
        padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 8),
        borderColor: AppColors.amber.withOpacity(0.7),
        child: Row(
          children: [
            const Icon(Icons.warning_amber, color: AppColors.amber, size: 16),
            const SizedBox(width: 4),
            Text('${robot.neglectStrikes}',
              style: text.titleSmall?.copyWith(color: AppColors.amber)),
          ],
        ),
      ),
    ],
  ],
);
}

class _StatCluster extends StatelessWidget {
  const _StatCluster({required this.robot});
  final RobotInstance robot;

```

```

@override
Widget build(BuildContext context) {
  return GlassPanel(
    padding: const EdgeInsets.fromLTRB(18, 18, 18, 20),
    borderColor: Colors.white.withOpacity(0.12),
    child: Column(
      children: [
        _HudStatBar(label: 'Battery', value: robot.battery.value,
          max: robot.battery.max, color: AppColors.green),
        const SizedBox(height: 16),
        _HudStatBar(label: 'Oil', value: robot.oil.value,
          max: robot.oil.max, color: AppColors.amber),
        const SizedBox(height: 16),
        _HudStatBar(label: 'Entertainment', value: robot.entertainment.value,
          max: robot.entertainment.max, color: AppColors.pink),
      ],
    ),
  );
}

class _HudStatBar extends StatelessWidget {
  const _HudStatBar({
    required this.label,
    required this.value,
    required this.max,
    required this.color,
  });

  final String label;
  final double value;
  final double max;
  final Color color;

  @override
  Widget build(BuildContext context) {
    final pct = (value / max).clamp(0.0, 1.0);
    final text = Theme.of(context).textTheme;

    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Row(
          children: [
            Text(label.toUpperCase(),
              style: text.titleSmall?.copyWith(letterSpacing: 1.5)),
            const Spacer(),
            Text('${(pct * 100).round()}%',
              style: text.titleSmall
                ?.copyWith(color: color, fontWeight: FontWeight.w700)),
          ],
        ),
        const SizedBox(height: 6),
        Container(
          height: 14,
          decoration: BoxDecoration(
            borderRadius: BorderRadius.circular(8),
            color: Colors.white.withOpacity(0.06),
            border: Border.all(color: color.withOpacity(0.35)),
          ),
          child: ClipRRect(
            borderRadius: BorderRadius.circular(8),
            child: Align(
              alignment: Alignment.centerLeft,
              child: TweenAnimationBuilder<double>(
                tween: Tween<double>(begin: 0, end: pct),
                duration: const Duration(milliseconds: 500),
                curve: Curves.easeOut,

```



```

        builder: (context, v, _) => FractionallySizedBox(
          widthFactor: v <= 0 ? 0.001 : v,
          child: Container(
            decoration: BoxDecoration(
              gradient: LinearGradient(
                colors: [color.withOpacity(0.65), color],
              ),
              boxShadow: [
                BoxShadow(color: color.withOpacity(0.6), blurRadius: 10),
              ],
            ),
          ),
        ),
      ),
    ),
  ],
);
}
}

class _NeonButton extends StatelessWidget {
  const _NeonButton({
    required this.label,
    required this.icon,
    required this.color,
    required this.onTap,
  });

  final String label;
  final IconData icon;
  final Color color;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: onTap,
      child: Container(
        padding: const EdgeInsets.symmetric(vertical: 14),
        decoration: BoxDecoration(
          borderRadius: BorderRadius.circular(14),
          gradient: LinearGradient(
            colors: [color.withOpacity(0.20), color.withOpacity(0.04)],
          ),
          border: Border.all(color: color.withOpacity(0.85), width: 1.4),
          boxShadow: [
            BoxShadow(
              color: color.withOpacity(0.35), blurRadius: 14, spreadRadius: -2),
          ],
        ),
        child: Column(
          children: [
            Icon(icon, color: color),
            const SizedBox(height: 4),
            Text(label.toUpperCase(),
              style: TextStyle(
                color: color,
                fontWeight: FontWeight.w600,
                letterSpacing: 1.5,
                fontSize: 12,
              ),
            ),
          ],
        ),
      ),
    );
  }
}

```

```

class _HudBackground extends StatelessWidget {
  const _HudBackground();

  @override
  Widget build(BuildContext context) {
    return DecoratedBox(
      decoration: const BoxDecoration(
        gradient: RadialGradient(
          center: Alignment(0, -0.35),
          radius: 1.1,
          colors: [Color(0xFF13243F), AppColors.bg],
        ),
      ),
      child: CustomPaint(painter: _GridPainter(), size: Size.infinite),
    );
  }
}

class _GridPainter extends CustomPainter {
  @override
  void paint(Canvas canvas, Size size) {
    final paint = Paint()
      ..color = Colors.cyan.withOpacity(0.05)
      ..strokeWidth = 1;
    const step = 34.0;
    for (double x = 0; x < size.width; x += step) {
      canvas.drawLine(Offset(x, 0), Offset(x, size.height), paint);
    }
    for (double y = 0; y < size.height; y += step) {
      canvas.drawLine(Offset(0, y), Offset(size.width, y), paint);
    }
  }

  @override
  bool shouldRepaint(covariant CustomPainter oldDelegate) => false;
}

```

5. Wire up the theme

In `lib/main.dart`, swap the old `ThemeData(...)` for the cyberpunk theme (everything else stays):

```

import 'ui/theme.dart';

// ...inside RoboPetApp.build:
return MaterialApp(
  title: 'RoboPet',
  debugShowCheckedModeBanner: false,
  theme: buildCyberpunkTheme(),
  home: const AppLifecycleReactor(child: PetScreen()),
);

```

flutter pub get, then flutter run. You will see: a neon grid backdrop, the breathing robot glowing on its platform in the center, glassmorphism stat bars with animated neon fills, and three glowing action buttons. The decay/feed logic is untouched, so everything still updates live.

6. Phase 2 -- Technical outline (Firebase, Gacha, Modules)

A brief approach before we dive in. Phase 2 turns the single local pet into a collection backed by a server-authoritative economy.

6.1 Firebase Auth & cloud sync

- Anonymous-first auth: `signInAnonymously()` on launch; replace the hardcoded local-player with the real uid. Offer account linking (Apple/Google) later so progress survives reinstalls.
- Authority split: Isar stays local-authoritative for care/decay (offline-first). Firestore is authoritative for the economy -- currencies, owned robots, owned modules, gacha pulls. A thin sync layer mirrors `PlayerProfile` to `users/{uid}` and hydrates Isar on login.
- Conflict rule: care stats reconcile via the existing timestamp engine; economy fields are only ever mutated server-side, so the client never wins a currency conflict.

6.2 Cloud Functions for Gacha (anti-cheat core)

- Callable function `rollGacha({ lootTableId, count })`. The client NEVER computes a draw. The function:
 - Reads the player doc in a Firestore transaction.
 - Validates currency (e.g. enough scrap/rareCogs); rejects otherwise.
 - Runs the weighted draw + pity server-side (e.g. epic guaranteed at pull 30), seeded server-side.
 - Writes new robot/module instances to inventory + decrements currency atomically.
 - Returns the resolved results for the client to animate.
- Loot tables live in Firestore/Remote Config so drop rates are tunable without an app release.
- Hardening: server timestamps only; never trust client time for rewards; reject premium-currency mutations that do not originate from a verified IAP/earn event.

6.3 Modules system

- Definition vs instance: `ModuleDef` (static content -- type, slot, stat modifiers, rarity) in Firestore; `ModuleInstance` in the player inventory (Isar + Firestore).
- Equip/unequip updates the robot Loadout slot (wheels/armor/sensor/core/utility) -- already in our schema.
- Derived stats: recompute on equip as `base + sum(flat)`, then `x(1 + sum(percent))`; never persist derived values (compute on read), so re-balancing is just a data change.
- UI: a collection/inventory grid + a module loadout screen, both reading the same reactive providers pattern we built in Phase 1.

6.4 Suggested build order

- Firebase project + anonymous auth + `PlayerProfile` <-> Firestore sync.
- Inventory & `ModuleDef`/`ModuleInstance` data model (extend Isar + Firestore).
- `rollGacha` Cloud Function + loot tables + currency validation.
- Gacha reveal animation (Flame) + collection grid.
- Module equip screen + derived-stat recompute.

When you are ready, the full Phase 2 TDD (schemas, the `rollGacha` function signature + transaction

logic, and the sync layer) can be produced as its own document -- and exported to PDF.